



Introdução à Lógica e Programação

Criando funções: parte 2

Variáveis locais

Variáveis locais são as variáveis definidas no corpo de uma função e os seus parâmetros. Variáveis locais são visíveis apenas no próprio corpo da função. O valor de uma variável local é reinicializado toda vez que a função é chamada.

```
def soma(a, b):  
    s = a + b  
    return s  
  
x, y = 5, 8  
sm = soma(x, y)  
print(sm)  
print(a) # gera erro  
print(b) # gera erro  
print(s) # gera erro
```

Como **s**, **a** e **b** são variáveis locais, elas não existem no programa principal.

Variáveis globais

Variáveis globais são as variáveis definidas no programa principal. Variáveis globais são visíveis para o programa principal bem como para as funções.

ATENÇÃO: o uso indiscriminado de variáveis globais aumenta a probabilidade de ocorrência de bugs em programas.

```
def soma(a):  
    return x + a  
  
x = 5  
y = soma(11)  
print(y) # 16
```

Neste exemplo **x** é
uma variável global.

Variáveis locais e globais com mesmo identificador

Uma atribuição de valor realizada no corpo de uma função faz com que a variável tenha escopo local, mesmo que a variável tenha identificador idêntico ao de uma variável global.

```
def soma(a, b):  
    s = a + b  
    return s
```

Para a função **soma**,
s é uma variável
local.

```
a, b, s = 5, 8, 0  
sm = soma(11, 4)  
print(sm) # 15  
print(a)  # 5  
print(b)  # 8  
print(s)  # 0
```

Instrução global

Permite que uma função altere o valor de uma variável global.

```
def muda_x():  
    global x  
    x = "Bola"  
  
x = "Casa"  
print(x)  # Casa  
muda_x()  
print(x)  # Bola
```

Instrução return e fluxo de execução

A instrução return encerra a execução de uma função. Acompanhe a execução do exemplo abaixo com o depurador.

```
def avaliar(x):  
    if x == 1:  
        return 1  
    return 0  
  
print(avaliar(1))  
print(avaliar(3))
```

Parâmetros opcionais

São parâmetros cujo valor não precisa ser informado no momento em que uma função é chamada. Como exemplo, temos a função `input` cujo parâmetro de mensagem é opcional.

```
a = input("Digite um nome")  
b = input()
```

Parâmetros opcionais e obrigatórios

Um parâmetro opcional é na verdade um parâmetro que possui um valor padrão. Caso um valor não seja informado durante a chamada da função, o parâmetro será usado com seu valor padrão. Um parâmetro obrigatório (parâmetro sem valor padrão) sempre deve ter seu valor definido durante a chamada da função. Parâmetros opcionais devem ser definidos após os parâmetros obrigatórios.

```
def linha(n, caractere='*'):
    print(caractere * n)

linha(10)
linha(15, '-')
linha()          # gera erro
```


Chamada com parâmetros nomeados

Em Python, ao chamar uma função podemos informar os valores dos parâmetros em conjunto, ou não, dos nomes dos parâmetros. Chamadas sem parâmetros nomeados seguem a ordem de parâmetros definida pela função.

```
def a_divisivel_por_b(a, b):  
    print(a % b == 0)  
  
a_divisivel_por_b(10, 2)           # True  
a_divisivel_por_b(2, 10)          # False  
a_divisivel_por_b(a=20, b=5)      # True  
a_divisivel_por_b(b=3, a=15)      # True  
a_divisivel_por_b(2, b=10)        # False  
a_divisivel_por_b(b=3, 15)        # Gera erro  
a_divisivel_por_b(2, a=3)         # Gera erro  
a_divisivel_por_b(2, a=3, b=5)    # Gera erro
```

Chamada com parâmetros nomeados

Parâmetros obrigatórios podem ter seus nomes omitidos, mas nesse caso deve-se seguir a ordem de definição dos parâmetros obrigatórios.

```
def a_divisivel_por_b(a, b, adorno=''):
    print(adorno, a % b == 0, adorno)

a_divisivel_por_b(10, 2)                # True
a_divisivel_por_b(2, 10)                # False
a_divisivel_por_b(2, 10, '*')           # False
a_divisivel_por_b(10, 2, adorno='*')    # * True *
a_divisivel_por_b(2, 10, adorno='*')    # * False *
a_divisivel_por_b(b=5, a=20)            # True
a_divisivel_por_b(a=15, b=3, adorno='*') # * True *
a_divisivel_por_b(10, adorno='*', b=5)   # * True *
a_divisivel_por_b(a=10, 2)              # Gera erro
```

Exercícios

1) Qual é saída do programa Python a seguir?

```
def funcao(a):  
    b = a * 2 + 1  
    return b  
  
b = 10  
a = funcao(b)  
print(a)  
print(b)
```

Exercícios

2) Qual é saída do programa Python a seguir?

```
def funcao(x):  
    if x == 0:  
        x = 1  
    y = 5 / x  
    return y  
  
x = 5  
y = funcao(x)  
print(x)  
print(y)  
y = -5  
funcao(y)  
print(x)  
print(y)
```

Exercícios

3) Qual é saída do programa Python a seguir?

```
def funcao(y):  
    if y < 0:  
        x = 1  
        return y  
    return y + 2  
  
x = 0  
w = funcao(10)  
print(x)  
print(w)  
w = funcao(-3)  
print(x)  
print(w)
```

Exercícios

4) Qual é saída do programa Python a seguir?

```
def funcao(y):  
    global x  
    if y < 0:  
        x = 1  
    return y  
    return y + 2  
  
x = 0  
w = funcao(10)  
print(x)  
print(w)  
w = funcao(-3)  
print(x)  
print(w)
```

Exercícios

5) Qual é saída do programa Python a seguir?

```
def funcao(i):  
    if -3 < i < -1:  
        return 'Z'  
    elif i == 0:  
        return 'Y'  
    x = i % 3  
    if x == 0:  
        return 'T'  
    elif x == 1:  
        return 'U'  
    return 'W'  
  
print(funcao(3))  
print(funcao(5))  
print(funcao(0))  
print(funcao(4))  
print(funcao(-10))  
print(funcao(-2))
```

Exercícios

6) Considerando as chamadas a seguir e a implementação apresentada no quadro, indique o valor de retorno da chamada de função ou se a chamada gera um erro.

- a) `funcao(1, 2)`
- b) `funcao(1, 2, 6)`
- c) `funcao(10)`
- d) `funcao(3, 2, d=10)`
- e) `funcao(3, 2, d=10, c=0)`
- f) `funcao(d=10, c=0, 2, 1)`
- g) `funcao(a=10, 0, 2, 1)`
- h) `funcao(a=10, d=3, c=1, b=5)`
- i) `funcao(5, b=3, d=1, c=2)`

```
def funcao(a, b, c=3, d=0):  
    return a + b + c + d
```


Referências

MENEZES, Nilo N. C.; **Introdução à Programação com Python**: Algoritmos e lógica de programação para iniciantes. 2ª ed, São Paulo: Novatec, 2014.

Informações bibliográficas

Autor: Alexandre Gomes de Lima

Data: setembro de 2025.

Direitos autorais: CC BY 4.0

(<https://creativecommons.org/licenses/by/4.0/deed.pt-br>)

